

UNIVERSIDAD ADVENTISTA DEL PLATA
FACULTAD DE CIENCIAS ECONÓMICAS Y DE LA ADMINISTRACIÓN
ALGORITMOS Y ESTRUCTURA DE DATOS

LISTA DE EJERCICIOS N° 5 — PROGRAMACIÓN ORIENTADA A OBJETOS (U5)

Sección 1: Clases, objetos, propiedades y métodos

1. Crear una clase `Campeon` que permita representar un personaje de un videojuego*.
 - a. La clase deberá tener las siguientes propiedades:
 - i. Nombre
 - ii. Nivel
 - iii. Puntos de vida
 - b. Crear la clase `Campeon`.
 - c. Crear un objeto de tipo `Campeon` y asignarle valores a sus propiedades.
 - d. Mostrar por consola los datos del campeón.
 - e. Crear un segundo objeto de tipo `Campeon` con valores diferentes y mostrar sus datos.
2. Agregar comportamiento a la clase `Campeon`*.
 - a. Modificar la clase del ejercicio anterior para incorporar los siguientes métodos:
 - i. `mostrarInformacion()`: muestra por consola el nombre, nivel y puntos de vida del campeón.
 - ii. `subirNivel()`: incrementa en 1 el nivel del campeón.
 - iii. `recibirDano(int cantidad)`: disminuye los puntos de vida en la cantidad recibida.
 - iv. Crear un campeón, mostrar su información, hacerlo subir de nivel y recibir daño. Mostrar nuevamente su información.
3. Crear una clase `Jugador` para representar a un jugador de un videojuego.
 - a. La clase deberá tener:
 - i. Nombre
 - ii. Nivel
 - iii. Puntos
 - b. Crear la clase y sus propiedades.
 - c. Crear un método `ganarPuntos(int cantidad)` que incremente los puntos del jugador.
 - d. Crear un método `subirNivel()` que incremente el nivel del jugador.
 - e. Crear dos objetos `Jugador` con diferentes valores.
 - f. Simular algunas acciones sobre los jugadores y mostrar su estado final.
4. Crear una clase `Producto` para representar un producto de una tienda.
 - a. La clase deberá tener:
 - i. Nombre
 - ii. Precio
 - iii. Stock

- b. Crear la clase y sus propiedades.
- c. Crear un método que permita aumentar el stock.
- d. Crear un método que permita disminuir el stock.
- e. Crear un método que muestre la información del producto.
- f. Crear algunos productos y realizar diferentes operaciones sobre ellos.

Sección 2: Objetos y estructuras de datos

5. Crear un ArrayList de objetos Campeon.
 - a. Utilizar la clase Campeon creada anteriormente.
 - b. Crear un ArrayList<Campeon>.
 - c. Agregar al menos 5 campeones.
 - d. Recorrer la lista y mostrar la información de cada campeón.
 - e. Buscar un campeón por su nombre.
 - f. Mostrar los campeones que tengan un nivel determinado o superior.
6. Modificar uno de los ejercicios realizados anteriormente en las listas de ejercicios para utilizar una clase.
 - a. Elegir uno de los siguientes ejercicios:
 - i. Triángulo.
 - ii. Jugador y KDA.
 - iii. Producto o promedio de calificaciones.
 - b. Crear una clase que represente la entidad principal del problema.
 - c. Incorporar al menos un método que permita realizar una operación relacionada con el problema.
 - d. Mostrar el resultado utilizando los objetos creados.

Sección 3: Herencia

7. Crear una clase Campeon y dos clases derivadas para representar diferentes tipos de personajes de un videojuego*.
 - a. Crear la clase Campeon con las propiedades:
 - i. Nombre
 - ii. Nivel
 - iii. Puntos de vida
 - b. Crear las clases CampeonFisico y CampeonMago, que hereden de Campeon.
 - c. Agregar a CampeonFisico una propiedad relacionada con su ataque físico.
 - d. Agregar a CampeonMago una propiedad relacionada con su poder mágico.
 - e. Crear al menos un objeto de cada tipo.
 - f. Mostrar por consola la información de cada campeón.
8. Modificar el ejercicio anterior para agregar comportamiento específico a cada tipo de campeón.
 - a. Crear en Campeon un método mostrarInformacion().
 - b. Hacer que CampeonFisico y CampeonMago tengan una implementación propia de dicho método.
 - c. Crear varios campeones de ambos tipos.
 - d. Mostrar la información de todos ellos.

Sección 4: Problemas integradores

9. Sistema básico de personajes para un videojuego.
 - a. Crear un programa que permita administrar los personajes de un videojuego.
 - b. Crear una clase Personaje con nombre, nivel y puntos de vida.
 - c. Crear al menos dos tipos de personajes mediante herencia.
 - d. Crear un ArrayList que permita almacenar los personajes.
 - e. Agregar diferentes personajes a la lista.
 - f. Recorrer la lista y mostrar la información de cada personaje.
 - g. Permitir realizar al menos una acción sobre los personajes, como subir de nivel o recibir daño.
10. Adaptar uno de los programas desarrollados anteriormente para separar responsabilidades mediante clases y métodos.
 - a. Identificar qué información debería pertenecer a una clase.
 - b. Crear la clase correspondiente.
 - c. Incorporar métodos para realizar las operaciones principales.
 - d. Utilizar objetos de dicha clase desde main.
 - e. Verificar que el programa mantenga el mismo comportamiento que la versión original.